



VISHWAKARMA
UNIVERSITY
Maximising Human Potential

TITLE OF PROJECT

Text2Speech Converter

Program :- B.Tech Technology

Course :- Technology Skill

Development - (Python)

Abbreviation :- TSDL

NAME: Snehal Late

INDEX

Sr No	Contents
1	Abstract
2	Introduction
3	Aim of project
4	Methodology
5	Flowchart & Implementation
6	Result
7	Conclusion

ABSTRACT

The "Text-to-Speech Converter with GUI" project is a Python application designed to facilitate the conversion of text into speech with a user-friendly graphical interface. Leveraging the gTTS (Google Text-to-Speech) library and the Tkinter GUI toolkit, this application allows users to input textual content and select a language or accent for the synthesized speech.

The core functionality involves the extraction of user-inputted text and language preferences, followed by the utilization of the gTTS library to generate corresponding speech. The graphical interface, implemented with Tkinter, provides an intuitive platform for users to interact with the application.

Key features of the project include error handling for input validation, ensuring that required details are provided before initiating the conversion process. Additionally, the application saves the synthesized speech as an MP3 file, enabling users to store and share the generated audio.

This project serves as a practical and accessible tool for individuals seeking a straightforward means to convert text into speech, with an emphasis on ease of use and a visually appealing user interface.

INTRODUCTION

In the digital age, the ability to convert text into natural-sounding speech has become an invaluable tool for various applications, from accessibility features to content creation. The "Text-to-Speech Converter with GUI" project addresses this need by providing a user-friendly interface for converting textual content into speech.

Developed in Python, this project leverages the gTTS (Google Text-to-Speech) library to seamlessly transform user-inputted text into synthesized speech. The graphical user interface (GUI), built using the Tkinter toolkit, ensures an intuitive and accessible experience for users, allowing them to easily interact with the application.

The primary objective of the project is to offer a straightforward solution for users to convert text to speech with minimal effort. The application prompts users to input the desired text and select a language or accent for the synthesized speech. Following input validation, the gTTS library generates the corresponding speech, which is then saved as an MP3 file for further use.

With an emphasis on simplicity and functionality, this project caters to individuals seeking an efficient and visually appealing text-to-speech conversion tool. Whether for personal use, educational purposes, or content creation, the "Text-to-Speech Converter with GUI" project provides a versatile solution for transforming text into audible content.

❖ AIM OF PROJECT :-

The aim of the "Text-to-Speech Converter with GUI" project is to create a user-friendly application that enables the conversion of text into natural-sounding speech. The primary objective of the project is to offer a straightforward solution for users to convert text to speech with minimal effort. The application prompts users to input the desired text and select a language or accent for the synthesized speech.

❖ METHODOLOGY :-

The development of the "Text-to-Speech Converter with GUI" project involves a systematic approach to ensure the creation of an efficient, user-friendly, and functional application. The methodology can be outlined as follows:

Requirement Analysis: Identify the primary requirements of the project, including the need for a text-to-speech conversion tool with a graphical user interface.

Define the essential features, such as user input for text and language preferences, audio file generation, and error handling.

Technology Stack Selection:

Choose the appropriate technologies for the project, considering the use of Python for scripting, the gTTS library for text-to-speech conversion, and Tkinter for GUI development.

Code Structuring:

Organize the code into modular components, defining functions for specific tasks such as text-to-speech conversion, input validation, and error handling.

Implement a clear and readable code structure, adhering to best practices and coding conventions.

Graphical User Interface Design:

Design the graphical user interface using Tkinter, ensuring an intuitive layout with input fields for text and language preferences.

Incorporate interactive elements such as buttons to trigger the text-to-speech conversion.

User Input Handling:

Implement mechanisms to capture user input for text and language preferences.

Perform input validation to ensure that required details are provided before initiating the conversion process.

Text-to-Speech Conversion:

Utilize the gTTS library to convert the user-provided text into synthesized speech.

Configure the language parameter based on user preferences to achieve natural-sounding speech.

Error Handling:

Implement error handling routines to gracefully manage situations where required details are not provided or if an error occurs during the text-to-speech conversion.

Display informative error messages to guide users in resolving issues.

Audio File Generation:

Save the synthesized speech as an MP3 file, allowing users to store and share the generated audio.

Verify the successful creation of the audio file.

Testing and Validation:

Conduct thorough testing to ensure the correctness and reliability of the application.

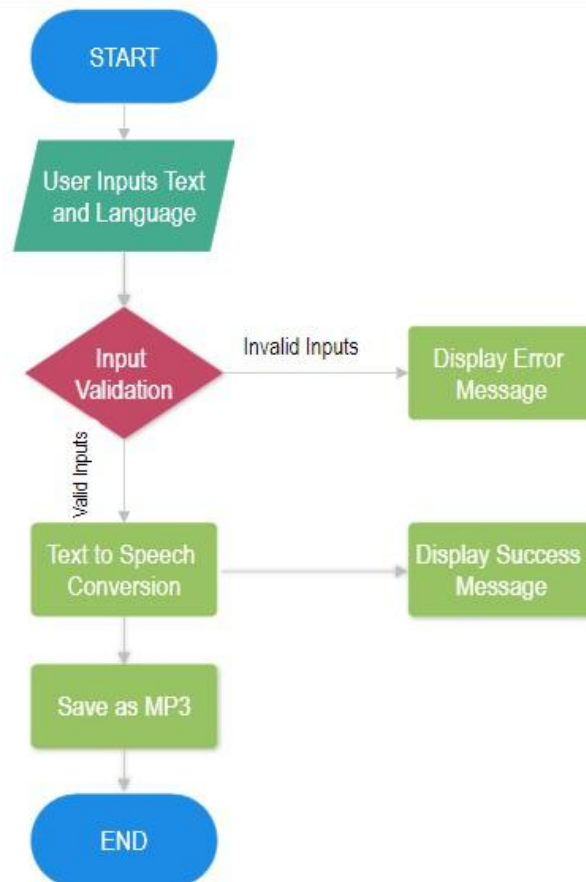
Validate the application's functionality across different scenarios, including valid and invalid inputs.

Documentation:

Prepare comprehensive documentation, including a README file explaining the usage, dependencies, and any additional instructions for users.

Document the code to enhance readability and provide insights into the implementation.

❖ FLOWCHART



❖ IMPLEMENTATION

Steps:

1. User inputs the desired text.
2. User selects a language or accent for the synthesized speech.
3. The gTTS library generates the corresponding speech.
4. The synthesized speech is saved as an MP3 file.

Code:

```
#Convert Speech to text and text to Speech
#import packages
from gtts import gTTS, lang
import os
from tkinter import *
from tkinter import messagebox
import speech_recognition as sr

#define functions
#text to speech conversion
def text_to_speech():
    #read inputs given by user
    text = text_entry.get("1.0", "end-1c")
    language = accent_entry.get()
    #Check if the user submitted inputs
    if (len(text)<=1) | (len(language)<=0):
        messagebox.showerror(message="Enter required details")
        return
    #Using the inputs, convert the text to speech
    speech = gTTS(text = text, lang = language, slow = False)
    #save the speech to an MP3 file
    speech.save("text.mp3")

#List the supported languages and their keys
def list_languages():
    #access languages and access codes using lang.tts_langs()
    messagebox.showinfo(message=list(lang.tts_langs().items()))
```

```

#speech to text conversion
def speech_to_text():
    # Initialize the recognizer class
    recorder = sr.Recognizer()

    try:
        duration = int(duration_entry.get())
    except ValueError:
        messagebox.showerror(message="Enter a valid duration (integer)")
        return

    # Use the microphone
    messagebox.showinfo(message="Speak into the microphone and wait after
finishing the recording")

    with sr.Microphone() as mic:
        # Prompt the user to record
        # Record audio from the user
        recorder.adjust_for_ambient_noise(mic)

        try:
            audio_input = recorder.listen(mic, timeout=duration)
            # Convert to text
            text_output = recorder.recognize_google(audio_input)
            # Display the output
            messagebox.showinfo(message="You said:\n " + text_output)
        except sr.UnknownValueError:
            messagebox.showwarning(message="No speech detected")
        except sr.RequestError as e:
            messagebox.showerror(message=f"Error with the speech recognition
service: {e}")

#Invoke call to class to view a window
window = Tk()
#Set dimensions of window and title
window.geometry("500x300")
window.title("Convert Speech to text and text to Speech: PythonGeeks")
title_label = Label(window, text="Convert Speech to text and text to Speech:
PythonGeeks").pack()
#Read inputs
#text_to_speech input
text_label = Label(window, text="Text:").place(x=10,y=20)
text_entry = Text(window, width=30,height=5)
text_entry.place(x=80,y=20)
#Accent input
accent_label = Label(window, text="Accent:").place(x=10,y=110)
accent_entry = Entry(window, width=26)
accent_entry.place(x=80,y=110)

```

```

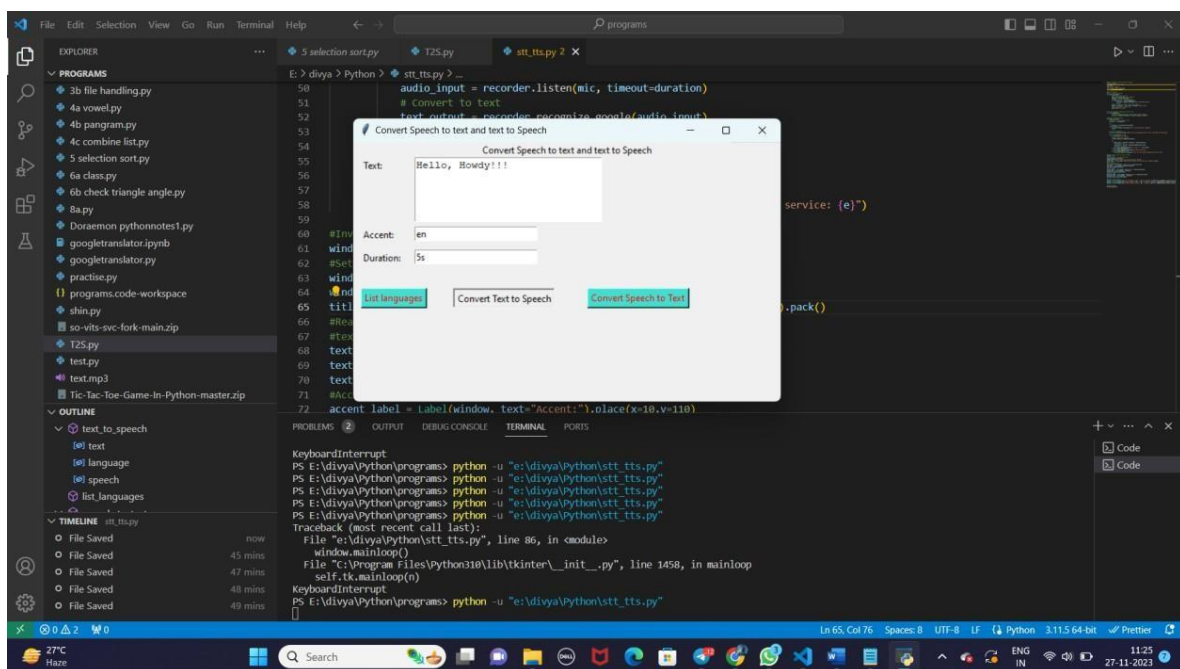
#Duration input
duration_label = Label(window, text="Duration:").place(x=10,y=140)
duration_entry = Entry(window, width=26)
duration_entry.place(x=80,y=140)

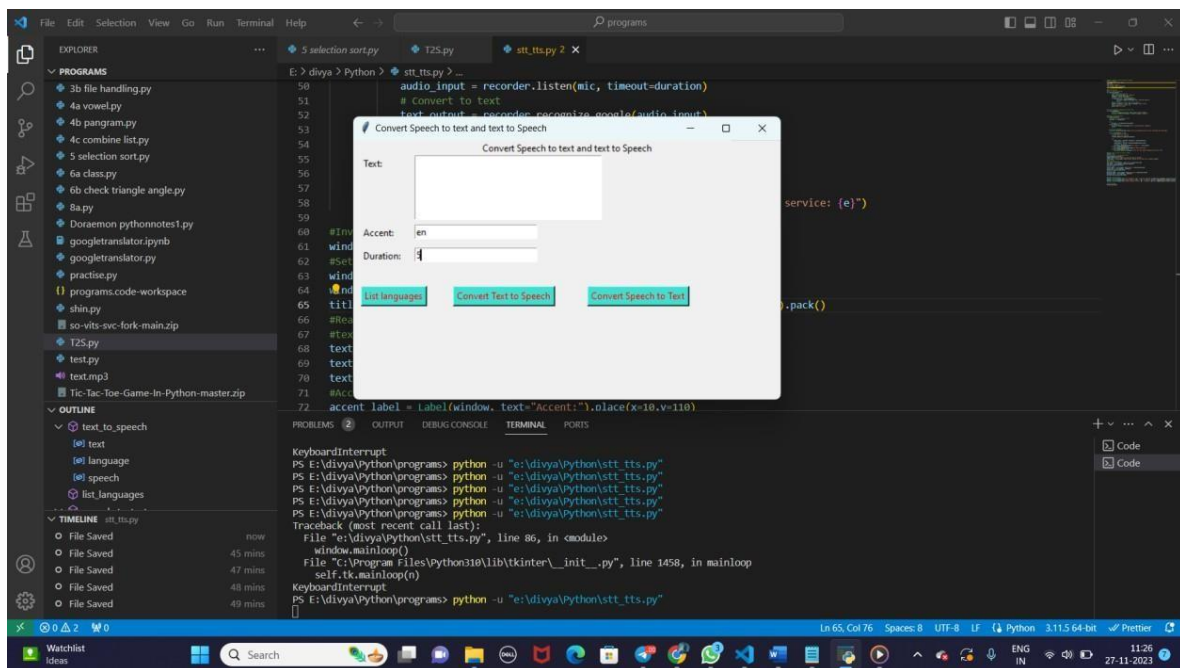
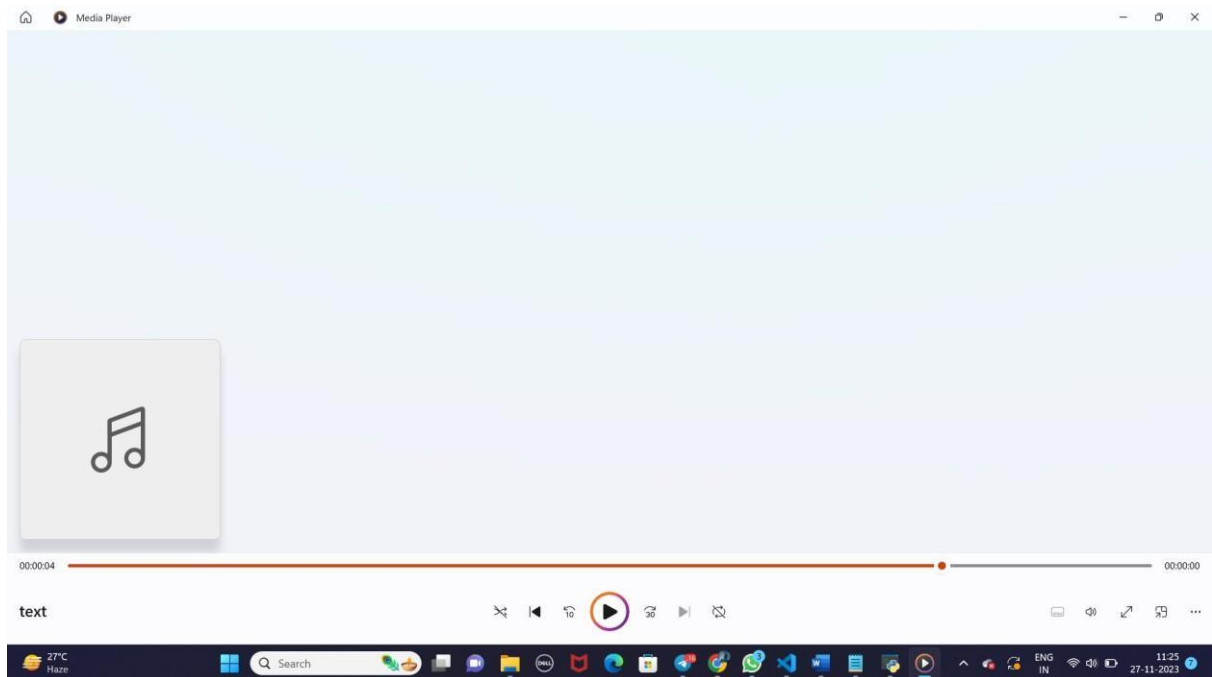
#Perform the functions
button1 = Button(window,text='List languages', bg =
'Turquoise',fg='Red',command=list_languages).place(x=10,y=190)
button2 = Button(window,text='Convert Text to Speech', bg =
'Turquoise',fg='Red',command=text_to_speech).place(x=130,y=190)
button3 = Button(window,text='Convert Speech to Text', bg =
'Turquoise',fg='Red',command=speech_to_text).place(x=305,y=190)

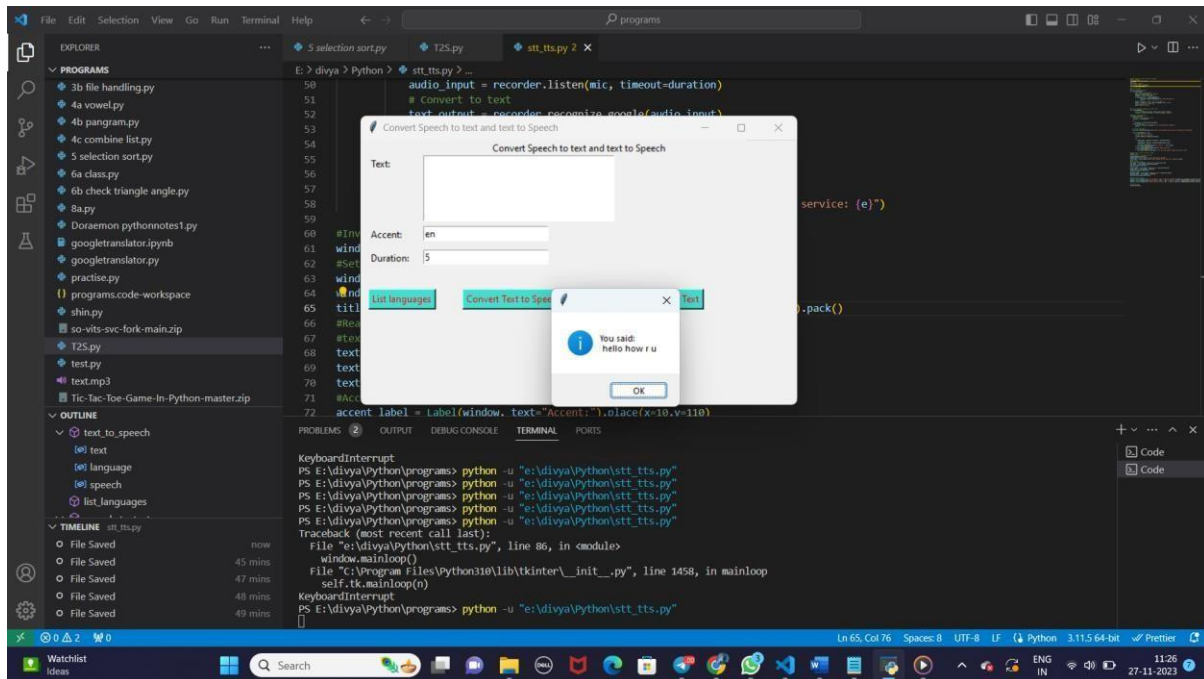
#close the app
window.mainloop()

```

■ RESULT :-







Conclusion :-

In conclusion, this project serves as a practical and accessible tool for individuals seeking a straightforward means to convert text into speech, with an emphasis on ease of use and a visually appealing user interface. The development process followed a structured methodology, resulting in an application that seamlessly integrates the gTTS library for text-to-speech conversion with the Tkinter GUI toolkit for an intuitive user interface.